

Lab 5: Geometric data decomposition implementation and analysis

1D block geometric data decomposition by columns

Section	Description
Geometric decomposition	1D block geometric data decomposition by columns
Code	The full code is the file: "mandel-omp-block-col.cpp" Basically we only modified the <i>mandel_simple</i> function. Firstly, we put the <i>#pragma omp parallel</i> from the beginning of the function until its end. Then, we created implicit tasks by obtaining the id of each thread and calculating the <i>start_it</i> and the <i>end_it</i> as explained in the theory: <pre> int my_id = omp_get_thread_num(); int howmany = omp_get_num_threads(); int bs = COLS / howmany; int remainder = COLS % howmany; int start_it, end_it; if (my_id < remainder) { start_it = my_id * (bs + 1); end_it = start_it + (bs + 1); } else { start_it = my_id * bs; end_it = start_it + bs; } </pre> The <i>remainder</i> part allowed us to achieve a better load balance and to not give all the extra iterations to the last thread. The <i>start_it</i> and the <i>end_it</i> are used in the most inner loop (the x one). Finally, we added the <i>#pragma omp atomic</i> and the <i>#pragma omp critical</i> to the usual parts of the Mandelbrot code.
Modelfactor analysis	• First table: At a certain moment the efficiency remains constant, that is a sign that something is not correct and that we are not taking all the profit from the parallelization. We have to observe that the elapsed time does not remain constant, and when we add more threads the time decreases, not a lot, but it's noticeable. That is directly related with the speedup, that it increases as we add more threads.

Overview of whole program execution metrics										
Number of threads	1	2	4	8	12	16	20	24	28	32
Elapsed time (sec)	3.24	2.60	2.21	1.44	1.02	0.75	0.63	0.56	0.50	0.43
Speedup	1.00	1.25	1.47	2.25	3.19	4.30	5.16	5.75	6.49	7.47
Efficiency	1.00	0.62	0.37	0.28	0.27	0.27	0.26	0.24	0.23	0.23

Table 1: Analysis done on Tue May 26 10:45:31 AM CEST 2026, par1104

• **Second table:**

The efficiency in the parallel region is about 99.85%, which is really good. Then, the global efficiency reaches the point of 24.02%, which makes sense because that is what we got previously. If we start to inspect the cause, we can see that the *Scalability for computation tasks* has a solid mark of 97.0%, so the problem is the *Parallelization strategy efficiency*, which reaches the poor mark of 24.75%. Inside the *Parallelization strategy efficiency* the thing that is causing the trouble is the Load balancing, which has the mark of 24.85%. This poor load balance can be explained by two factors. The first is that when each thread enters at the function *pixel_dwell* it can happen that some finish earlier ($\text{lengthsq} < (N * N)$) because they calculate an exterior point, while others calculate an inner point ($\text{dwell} < \text{maxiter}$) and take much more time and iterations to finish. Secondly, it can happen that one cache line can be occupied by two threads, so when this happens they crush the data from the other thread causing a lot of cache misses and losing time, while other threads might have a lot less cache misses.

Overview of the Efficiency metrics in parallel fraction, $\phi=99.58\%$										
Number of threads	1	2	4	8	12	16	20	24	28	32
Global efficiency	100.00%	62.31%	36.80%	28.29%	26.85%	27.27%	26.24%	24.43%	23.75%	24.02%
Parallelization strategy efficiency	100.00%	62.32%	36.83%	28.34%	26.97%	27.51%	26.68%	25.04%	24.56%	24.75%
Load balancing	100.00%	62.33%	36.84%	28.36%	26.99%	27.55%	26.72%	25.10%	24.64%	24.85%
In execution efficiency	100.00%	99.99%	99.98%	99.94%	99.92%	99.85%	99.81%	99.75%	99.67%	99.61%
Scalability for computation tasks	100.00%	99.98%	99.92%	99.82%	99.58%	99.13%	98.38%	97.59%	96.68%	97.05%
IPC scalability	100.00%	100.04%	100.02%	99.98%	99.80%	99.49%	98.92%	98.60%	97.51%	98.15%
Instruction scalability	100.00%	100.00%	100.00%	100.00%	99.99%	99.99%	99.99%	99.63%	99.98%	99.98%
Frequency scalability	100.00%	99.94%	99.90%	99.85%	99.78%	99.65%	99.46%	99.34%	99.16%	98.90%

Table 2: Analysis done on Tue May 26 10:45:31 AM CEST 2026, par1104

• **Third table:**

Each thread only executes one implicit task, which is what we expected. The useful duration of implicit tasks is being reduced thanks to the fact that each thread makes less iterations. The load balancing stays constant with the mark 0.25 when arriving at 8 threads, this value is really low. And as we are not using the *atomic* and *critical* to make these tables then the Synch% is always 0. The time in fork/join increases a lot because there are more tasks to create (one per thread) and because some tasks take much longer to finish and therefore the 'join' takes place later. Some tasks take longer to finish from the load imbalance that we have talked about before.

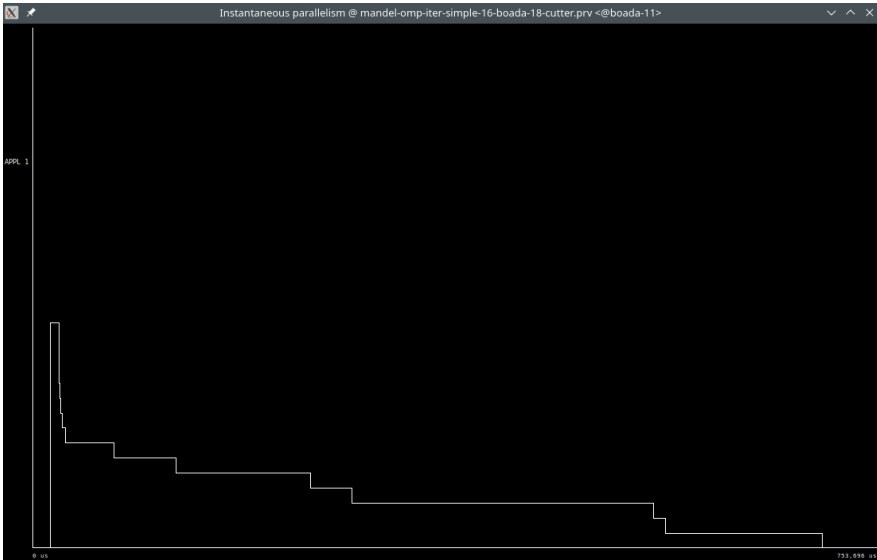
	Overheads in executing implicit tasks									
Number of threads	1	2	4	8	12	16	20	24	28	32
Number of implicit tasks per thread (average us)	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0
Useful duration for implicit tasks (average us)	3227834.04	1614183.84	807633.97	404187.5	270133.63	203509.21	164055.68	137817.13	119244.43	103933.68
Load balancing for implicit tasks	1.0	0.62	0.37	0.28	0.27	0.28	0.27	0.25	0.25	0.25
Time in synchronization implicit tasks (average us)	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Time in fork/join implicit tasks (average us)	108.91	159.28	1793893.82	1404265.79	988055.11	729100.26	605775.99	542439.63	478082.54	414231.07

Table 3: Analysis done on Tue May 26 10:45:31 AM CEST 2026, par1104

Paraver analysis

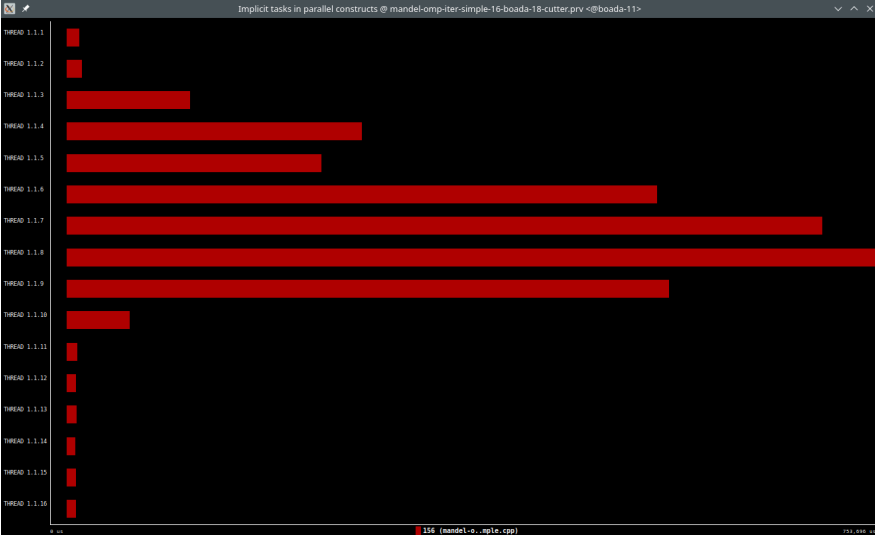
• Hint Useful/Instantaneous Parallelism:

The next plot tells us that at the beginning there is a lot of parallelization, as tasks are created at the same time entering the parallel section. Then, the parallelization starts to decay, and that happens from the load imbalance that we mentioned, some tasks finish much earlier because they calculate a block of outer points.



• Hint OpenMP/Implicit tasks in parallel constructs:

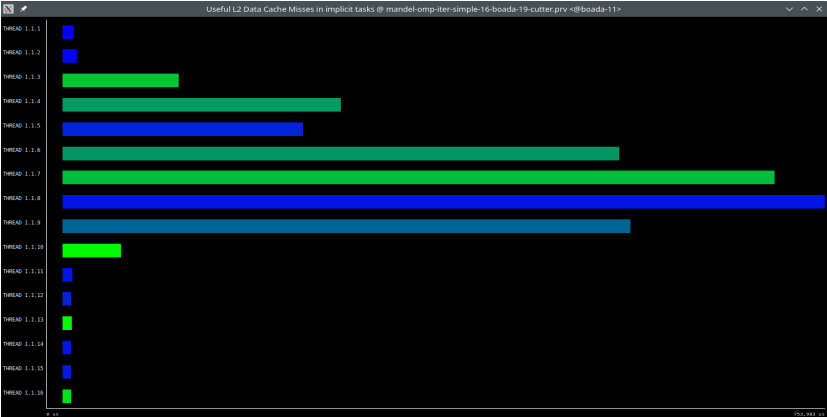
The following plot shows graphically the load imbalance, because some threads like the 6th, 7th, 8th or 9th spend a lot of time making calculations, whereas some other threads finish a lot earlier. This also explains the fork/join overhead seen before.



Memory analysis

THREAD 1.1.1	200,524
THREAD 1.1.2	197,319
THREAD 1.1.3	123,071
THREAD 1.1.4	140,719
THREAD 1.1.5	186,031
THREAD 1.1.6	141,743
THREAD 1.1.7	126,122
THREAD 1.1.8	192,841
THREAD 1.1.9	161,138
THREAD 1.1.10	103,203
THREAD 1.1.11	192,798
THREAD 1.1.12	185,523
THREAD 1.1.13	103,110
THREAD 1.1.14	192,428
THREAD 1.1.15	189,726
THREAD 1.1.16	113,512
Total	2,549,808
Average	159,363
Maximum	200,524
Minimum	103,110
StDev	35,609.20
Avg/Max	0.79

The figure from below shows the same as the previous one, the duration of each task. Then with the histogram we can say that the number of cache misses is not related with how long the task is, because thread 1 and thread 8 have similar cache misses. Then, we can deduce that no matter how long the task is, cache misses are directly related with the fact that all the columns they treat fit in a cache line or not. If not, both threads share a cache line and this produces the cache misses. The case that we would want is the one when each thread approximately has the same number of cache misses.

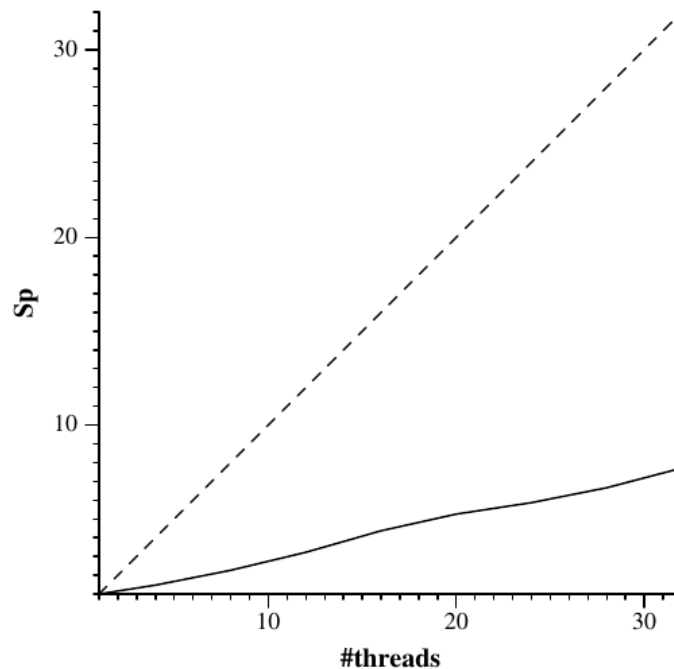


Then we have the l2_misses.txt file, which is:

Num threads	Fallades Cache L2	Mitjana de fallades
1	1613737	1613737
2	1723244	861622
4	1906359	476589
8	2226229	278278
12	2617628	218135
16	2557381	159836
20	2827554	141377
24	2908162	121173
28	3002212	107221
32	1980907	61903

We can see how adding more threads increases the cache misses, even we duplicate them by going from one thread to twenty eight. This can happen because as more threads the probability of two threads sharing the same cache line increases. We would want that when adding more threads we get the same cache misses or just a few more.

Strong scalability



par1104

Speed-up wrt sequential time (mandel function only)

Generated by par1104 on Tue May 26 11:20:55 AM CEST 2026

Finally we have the strong scalability plot. When analyzing it, we can see that even if the speedup is growing as there are

	more threads, is far from the ideal one. This is due to the fact that the load balance is not the best one as we discussed before.
--	--

1D block-cyclic geometric data decomposition by columns

Section	Description
Geometric decomposition	1D block-cyclic geometric data decomposition by columns
Code	The full code is in the file: "blockcyclic.cpp" The most important parts of the code are: · First we open a parallel region with: <code>#pragma omp parallel</code> This means that all the threads will execute the code inside this block. Then each thread obtains its own identifier and the total number of threads: <pre>int my_id = omp_get_thread_num(); int howmany = omp_get_num_threads();</pre> We use these values to manually distribute the work between the threads. After that, we define the block size: <pre>int BS = 64 / sizeof(int);</pre> This is an important part of the code. The value 64 represents the size of a cache line in bytes. So we divide of the size of integer his helps each thread work with consecutive elements in memory and improves cache locality. The work distribution is: <pre>for (int xx = my_id * BS; xx < COLS; xx += howmany * BS)</pre> Each thread starts in a different block depending on its thread id. Then it jumps by <code>howmany * BS</code>, so two threads do not calculate the same. Inside each loop, the thread computes one by one <pre>for (int x = xx; x < xx + BS && x < COLS; x++)</pre>

	The condition $x < \text{COLS}$ is needed because the last block may go outside. Later, we protect the histogram with <code>pragma omp</code> to avoid the data race condition <pre>#pragma omp atomic histogram[M[y][x] - 1]++;</pre> Finally we do a critical because the functions of above aren't simple arithmetic, and we do this to avoid data race condition <pre>#pragma omp critical { XSetForeground(display, gc, color); XDrawPoint(display, win, gc, x, y); }</pre>																																																							
Modelfact or analysis	<table><tr><th colspan="11">Overview of whole program execution metrics</th></tr><tr><th>Number of threads</th><td>1</td><td>2</td><td>4</td><td>8</td><td>12</td><td>16</td><td>20</td><td>24</td><td>28</td><td>32</td></tr><tr><th>Elapsed time (sec)</th><td>3.25</td><td>1.63</td><td>0.83</td><td>0.43</td><td>0.29</td><td>0.23</td><td>0.19</td><td>0.16</td><td>0.14</td><td>0.14</td></tr><tr><th>Speedup</th><td>1.00</td><td>1.99</td><td>3.92</td><td>7.62</td><td>11.23</td><td>14.23</td><td>17.39</td><td>20.59</td><td>22.82</td><td>23.48</td></tr><tr><th>Efficiency</th><td>1.00</td><td>0.99</td><td>0.98</td><td>0.95</td><td>0.94</td><td>0.89</td><td>0.87</td><td>0.86</td><td>0.82</td><td>0.73</td></tr></table> Table 1: Analysis done on Wed May 27 04:49:14 PM CEST 2026, par1119 Table 1: The table shows that the elapsed time decreases as the number of threads increases. The speedup is calculated as the time with 1 thread divided by the time with N threads. The efficiency is calculated as the speedup divided by the number of threads. We can see that the speedup increases almost linearly at the beginning, especially from 1 to 12 threads. However, when the number of threads becomes higher, the efficiency starts to decrease. This happens because there is more overhead, synchronization cost, and memory contention. From 28 to 32 threads, the elapsed time remains the same, 0.14 seconds. This means that adding more threads does not improve the execution time anymore. Therefore, for this version, using around 28 threads is enough, because 32 threads gives the same time but worse efficiency.	Overview of whole program execution metrics											Number of threads	1	2	4	8	12	16	20	24	28	32	Elapsed time (sec)	3.25	1.63	0.83	0.43	0.29	0.23	0.19	0.16	0.14	0.14	Speedup	1.00	1.99	3.92	7.62	11.23	14.23	17.39	20.59	22.82	23.48	Efficiency	1.00	0.99	0.98	0.95	0.94	0.89	0.87	0.86	0.82	0.73
Overview of whole program execution metrics																																																								
Number of threads	1	2	4	8	12	16	20	24	28	32																																														
Elapsed time (sec)	3.25	1.63	0.83	0.43	0.29	0.23	0.19	0.16	0.14	0.14																																														
Speedup	1.00	1.99	3.92	7.62	11.23	14.23	17.39	20.59	22.82	23.48																																														
Efficiency	1.00	0.99	0.98	0.95	0.94	0.89	0.87	0.86	0.82	0.73																																														

Overview of the Efficiency metrics in parallel fraction, $\phi=99.60\%$										
Number of threads	1	2	4	8	12	16	20	24	28	32
Global efficiency	100.00%	99.81%	99.43%	98.43%	97.46%	93.74%	93.15%	92.83%	89.26%	80.43%
Parallelization strategy efficiency	100.00%	99.95%	99.71%	98.80%	97.97%	94.45%	94.13%	94.18%	90.86%	82.28%
Load balancing	100.00%	99.96%	99.75%	98.96%	98.29%	94.96%	94.82%	95.04%	92.02%	83.39%
In execution efficiency	100.00%	99.99%	99.96%	99.85%	99.67%	99.46%	99.27%	99.09%	98.75%	98.67%
Scalability for computation tasks	100.00%	99.87%	99.72%	99.62%	99.48%	99.24%	98.96%	98.57%	98.23%	97.75%
IPC scalability	100.00%	99.93%	99.84%	99.80%	99.71%	99.58%	99.44%	99.28%	99.15%	98.84%
Instruction scalability	100.00%	100.00%	100.00%	100.00%	99.99%	99.99%	99.99%	99.99%	99.98%	99.98%
Frequency scalability	100.00%	99.93%	99.88%	99.83%	99.78%	99.67%	99.53%	99.30%	99.09%	98.92%

Table 2: Analysis done on Wed May 27 04:49:14 PM CEST 2026, par1119

Table 2: Analyzing the second table, we can see that the parallel fraction is very high, 99.60%. This means that almost all the program has been parallelized, so the code can take good advantage of using several threads.

Most of the efficiency metrics are close to 100%, especially the scalability and instruction scalability values. This means that the program scales quite well when we increase the number of threads. However, the global efficiency decreases when the number of threads increases. For example, with 1 thread it is 100%, but with 32 threads it goes down to 80.43%. The main reason is the load balancing efficiency, which also decreases a lot, from 100% to 83.39%.

This means that the work is not perfectly balanced between all threads. Some threads may finish earlier, while others spend more time computing their blocks. Because of this, the total efficiency gets worse with many threads.

In conclusion, the program has a very good parallel fraction and good scalability, but the main problem is load imbalance when using a high number of threads.

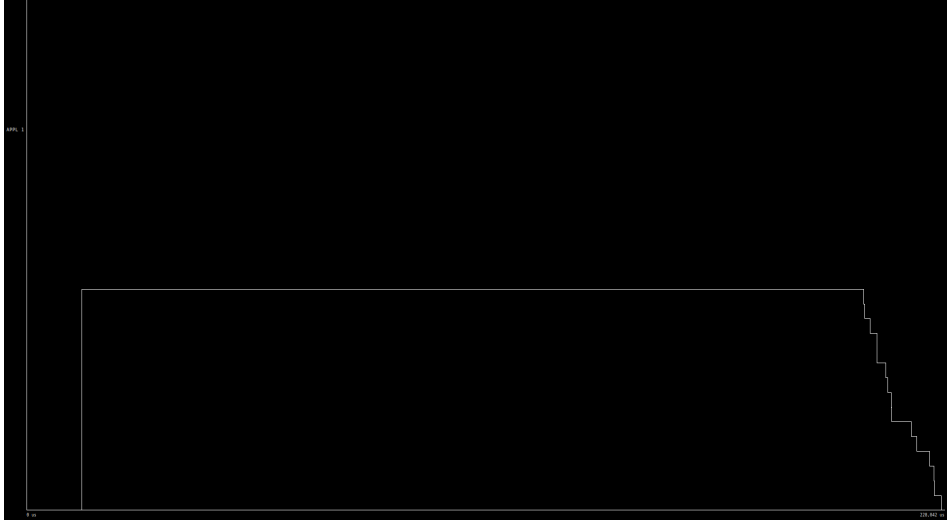
Overheads in executing implicit tasks										
Number of threads	1	2	4	8	12	16	20	24	28	32
Number of implicit tasks per thread (average us)	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0
Useful duration for implicit tasks (average us)	3232303.66	1618306.37	810348.9	405578.81	270763.42	203563.75	163321.04	136634.6	117514.97	103334.46
Load balancing for implicit tasks	1.0	1.0	1.0	0.99	0.98	0.95	0.95	0.95	0.92	0.83
Time in synchronization implicit tasks (average us)	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Time in fork/join implicit tasks (average us)	114.31	1541.86	3423.31	5570.53	1520.08	3672.53	3250.34	5982.79	5964.79	30533.54

Table 3: Analysis done on Wed May 27 04:49:14 PM CEST 2026, par1119

Table 3:

Analyzing the third table, we can see the overheads in executing implicit tasks.

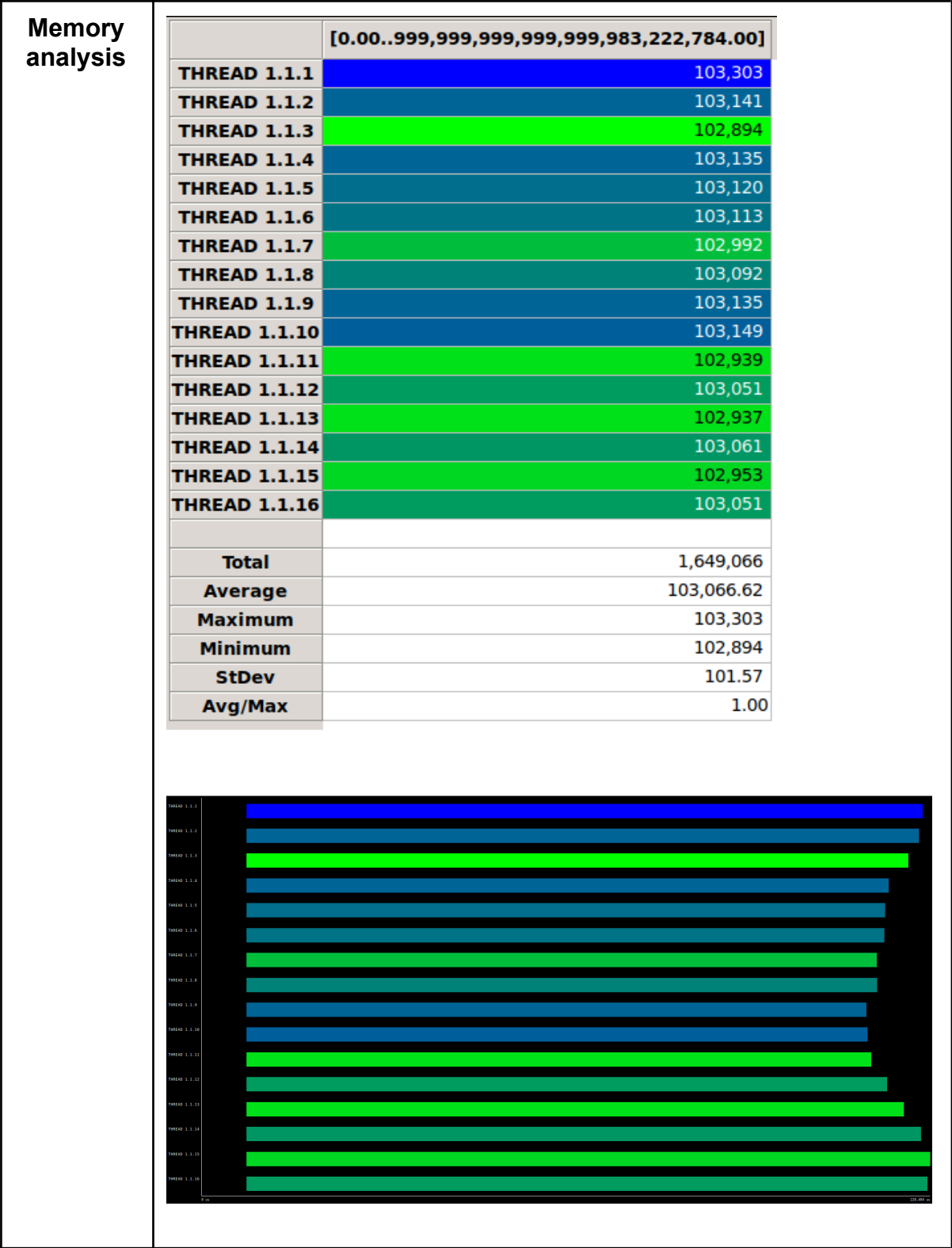
The number of implicit tasks per thread is always 1.0, which means that each thread executes one implicit task inside the parallel region. The useful duration of the implicit tasks decreases when the number of threads increases. For example, with 1 thread it is 3232303.66,

	while with 32 threads it decreases to 103334.46. This is expected, because the work is divided between more threads. The load balancing is very good at the beginning, close to 1.0. However, it decreases when using many threads, reaching 0.83 with 32 threads. This means that with more threads the work is not perfectly balanced, and some threads finish earlier than others. The synchronization time is 0.0 in all cases, so synchronization is not a problem in this version. Finally, the fork/join time increases with more threads. This is normal because creating and joining more threads has an extra cost.
Paraver analysis	 Hint Useful/Instantaneous Parallelism: This graph shows that after a short initial phase, the useful parallelism becomes stable for most of the execution. This means that most threads are working at the same time during the main part of the program. However, near the end of the execution, the parallelism starts to decrease step by step. This means that some threads finish their work earlier, while other threads are still computing. As a result, some threads become idle. This confirms the load imbalance that we saw in the previous tables. The program has good parallelism during most of the execution, but at the end the work is not perfectly balanced between all threads.



Hint OpenMP/Implicit tasks in parallel constructs: This plot shows the execution time of the implicit tasks for each thread. In this case, we can see that most threads work for almost the same amount of time, so the load balance is quite good during most of the execution.

However, there are still small differences between threads. Some threads finish a little earlier than others, while the last threads continue working for slightly more time. This explains why the load balancing is not perfect in the previous tables.



Num threads	Fallades Cache L2	Mitjana de fallades
1	1612254	1612254
2	3171898	1585949
4	4609511	1152377
8	1648431	206053
12	1652529	137710
16	1655161	103447
20	1658731	82936
24	1661543	69230
28	1664513	59446
32	1663407	51981

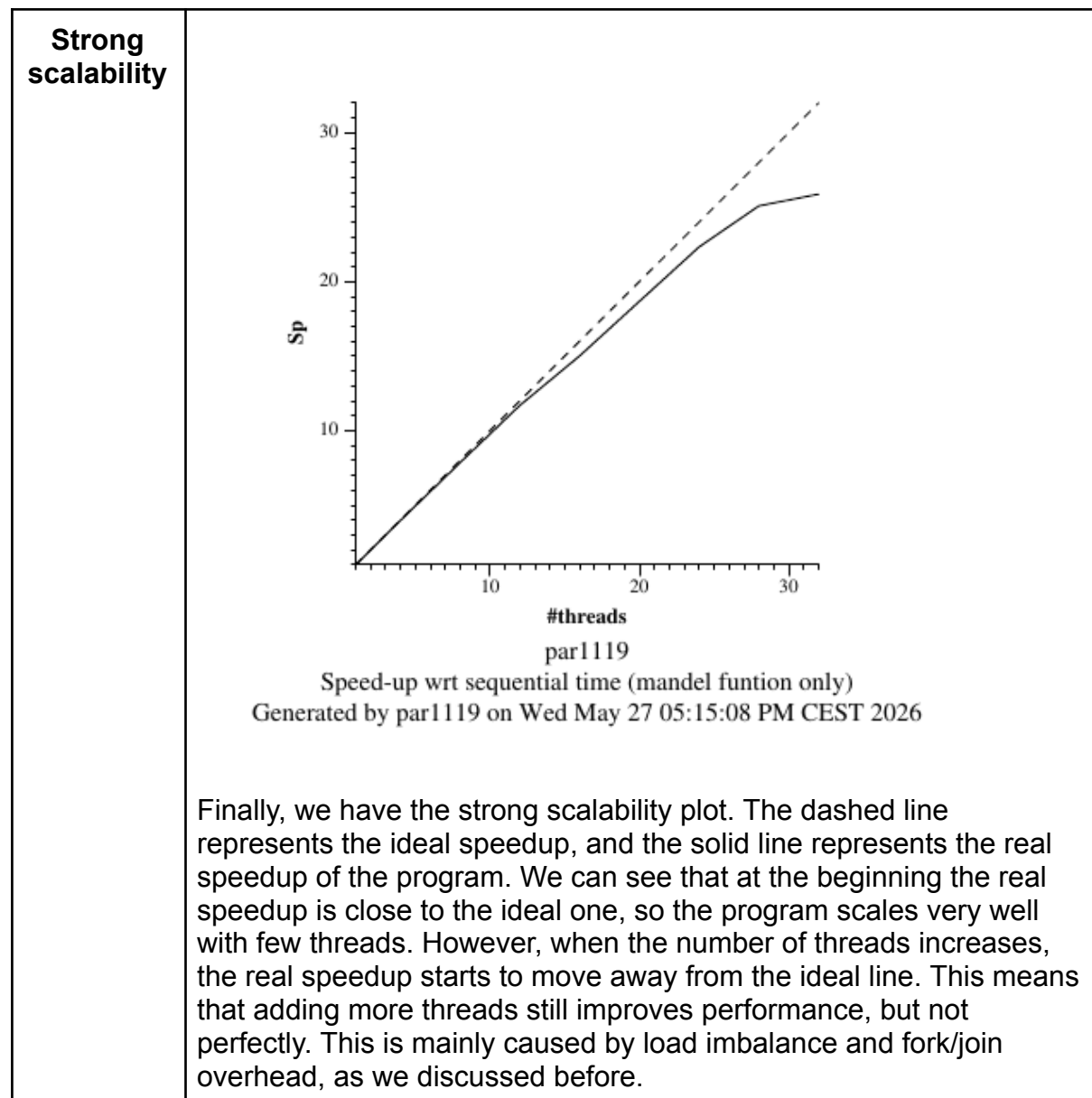
This analysis shows the cache misses of the program. The table has three columns: number of threads, total cache misses, and average cache misses per thread.

We can see that after 8 threads, the total number of cache misses stays almost constant, 1,65 million. This is good because using more threads does not create many extra cache misses.

The average cache misses per thread decreases when the number of threads increases. This is expected because the work is divided between more threads.

The Paraver table shows one execution with 16 threads. All threads have around 103,000 cache misses, with a very small difference between the maximum and the minimum. This means that the cache misses are very well balanced between threads.

So, the load imbalance of the program is probably not caused by cache misses, because all threads have very similar cache behavior.



1D cyclic geometric data decomposition by rows

Section	Description
Geometric decomposition	1D cyclic geometric data decomposition by rows
Code	The full code is in the file: "mandel-omp-cyc-row.cpp" Here we just had to change a few things from the original code. We added the <code>#pragma omp parallel</code> as usual, then, we obtained the id of each thread and how many of them are. We modified the outer loop (the y one) so that <code>y = my_id</code> and the y increased by <code>howmany</code> each iteration:

	<pre>int my_id = omp_get_thread_num(); int howmany = omp_get_num_threads(); // Calcular for (int y = my_id; y < ROWS; y += howmany)</pre> Finally, we added the <code>#pragma omp atomic</code> and <code>#pragma omp critical</code> to the same parts of code as always.																																																																																																																																																																					
Modelfactor analysis	• First table: The first table shows us some really good numbers. First of all, we see that the elapsed time is really low when arriving at 32 threads. Then, the efficiency grows in a significant way, we get to the mark of 27.33, which is really good. The speedup stays solid around the 0.8 and 0.9 mark, which is a sign that the load imbalance is not causing trouble. <table><tr><th colspan="11">Overview of whole program execution metrics</th></tr><tr><th>Number of threads</th><th>1</th><th>2</th><th>4</th><th>8</th><th>12</th><th>16</th><th>20</th><th>24</th><th>28</th><th>32</th></tr><tr><td>Elapsed time (sec)</td><td>3.22</td><td>1.62</td><td>0.82</td><td>0.42</td><td>0.28</td><td>0.22</td><td>0.18</td><td>0.15</td><td>0.13</td><td>0.12</td></tr><tr><td>Speedup</td><td>1.00</td><td>1.99</td><td>3.94</td><td>7.75</td><td>11.43</td><td>14.93</td><td>18.35</td><td>21.49</td><td>24.51</td><td>27.33</td></tr><tr><td>Efficiency</td><td>1.00</td><td>0.99</td><td>0.99</td><td>0.97</td><td>0.95</td><td>0.93</td><td>0.92</td><td>0.90</td><td>0.88</td><td>0.85</td></tr></table> Table 1: Analysis done on Tue May 26 11:36:03 AM CEST 2026, par1104 • Second table: Then, in the second table we can observe that overall the numbers stay around 99% or higher. All we can say is that the efficiency of the parallel fraction is almost 100% and that the global efficiency even with 32 threads is 95.51%. But we think that the most important thing to notice is the fact that the load balancing has a great value of 99.45%. This means that the load imbalance that we saw for example in the first strategy has disappeared. The load imbalance might have disappeared because now each thread does not just make a big block (which could be a block of full inner points), now we get a row and we process it, then another and so on. With this new way of parallelization we have more chances that a thread does not make just inner or outer points, now each one does a little bit of everything. <table><tr><th colspan="11">Overview of the Efficiency metrics in parallel fraction, $\phi=99.60\%$</th></tr><tr><th>Number of threads</th><th>1</th><th>2</th><th>4</th><th>8</th><th>12</th><th>16</th><th>20</th><th>24</th><th>28</th><th>32</th></tr><tr><td>Global efficiency</td><td>100.00%</td><td>99.86%</td><td>99.75%</td><td>99.52%</td><td>99.35%</td><td>98.80%</td><td>98.36%</td><td>97.39%</td><td>96.61%</td><td>95.51%</td></tr><tr><td>Parallelization strategy efficiency</td><td>100.00%</td><td>99.95%</td><td>99.88%</td><td>99.73%</td><td>99.62%</td><td>99.35%</td><td>99.21%</td><td>98.66%</td><td>98.15%</td><td>97.73%</td></tr><tr><td>Load balancing</td><td>100.00%</td><td>99.97%</td><td>99.93%</td><td>99.90%</td><td>99.93%</td><td>99.85%</td><td>99.89%</td><td>99.71%</td><td>99.59%</td><td>99.45%</td></tr><tr><td>In execution efficiency</td><td>100.00%</td><td>99.99%</td><td>99.95%</td><td>99.83%</td><td>99.69%</td><td>99.50%</td><td>99.31%</td><td>98.95%</td><td>98.55%</td><td>98.27%</td></tr><tr><td>Scalability for computation tasks</td><td>100.00%</td><td>99.91%</td><td>99.86%</td><td>99.80%</td><td>99.74%</td><td>99.45%</td><td>99.15%</td><td>98.71%</td><td>98.43%</td><td>97.73%</td></tr><tr><td>IPC scalability</td><td>100.00%</td><td>99.98%</td><td>100.00%</td><td>99.97%</td><td>99.96%</td><td>99.79%</td><td>99.64%</td><td>99.40%</td><td>99.24%</td><td>98.87%</td></tr><tr><td>Instruction scalability</td><td>100.00%</td><td>100.00%</td><td>100.00%</td><td>100.00%</td><td>100.00%</td><td>100.00%</td><td>100.00%</td><td>100.00%</td><td>100.00%</td><td>100.00%</td></tr><tr><td>Frequency scalability</td><td>100.00%</td><td>99.92%</td><td>99.86%</td><td>99.83%</td><td>99.78%</td><td>99.65%</td><td>99.50%</td><td>99.30%</td><td>99.18%</td><td>98.85%</td></tr></table> Table 2: Analysis done on Tue May 26 11:36:03 AM CEST 2026, par1104 •Third table: With the third table we can confirm what we knew from before, the load balance is extremely good. With 32 threads	Overview of whole program execution metrics											Number of threads	1	2	4	8	12	16	20	24	28	32	Elapsed time (sec)	3.22	1.62	0.82	0.42	0.28	0.22	0.18	0.15	0.13	0.12	Speedup	1.00	1.99	3.94	7.75	11.43	14.93	18.35	21.49	24.51	27.33	Efficiency	1.00	0.99	0.99	0.97	0.95	0.93	0.92	0.90	0.88	0.85	Overview of the Efficiency metrics in parallel fraction, $\phi=99.60\%$											Number of threads	1	2	4	8	12	16	20	24	28	32	Global efficiency	100.00%	99.86%	99.75%	99.52%	99.35%	98.80%	98.36%	97.39%	96.61%	95.51%	Parallelization strategy efficiency	100.00%	99.95%	99.88%	99.73%	99.62%	99.35%	99.21%	98.66%	98.15%	97.73%	Load balancing	100.00%	99.97%	99.93%	99.90%	99.93%	99.85%	99.89%	99.71%	99.59%	99.45%	In execution efficiency	100.00%	99.99%	99.95%	99.83%	99.69%	99.50%	99.31%	98.95%	98.55%	98.27%	Scalability for computation tasks	100.00%	99.91%	99.86%	99.80%	99.74%	99.45%	99.15%	98.71%	98.43%	97.73%	IPC scalability	100.00%	99.98%	100.00%	99.97%	99.96%	99.79%	99.64%	99.40%	99.24%	98.87%	Instruction scalability	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	Frequency scalability	100.00%	99.92%	99.86%	99.83%	99.78%	99.65%	99.50%	99.30%	99.18%	98.85%
Overview of whole program execution metrics																																																																																																																																																																						
Number of threads	1	2	4	8	12	16	20	24	28	32																																																																																																																																																												
Elapsed time (sec)	3.22	1.62	0.82	0.42	0.28	0.22	0.18	0.15	0.13	0.12																																																																																																																																																												
Speedup	1.00	1.99	3.94	7.75	11.43	14.93	18.35	21.49	24.51	27.33																																																																																																																																																												
Efficiency	1.00	0.99	0.99	0.97	0.95	0.93	0.92	0.90	0.88	0.85																																																																																																																																																												
Overview of the Efficiency metrics in parallel fraction, $\phi=99.60\%$																																																																																																																																																																						
Number of threads	1	2	4	8	12	16	20	24	28	32																																																																																																																																																												
Global efficiency	100.00%	99.86%	99.75%	99.52%	99.35%	98.80%	98.36%	97.39%	96.61%	95.51%																																																																																																																																																												
Parallelization strategy efficiency	100.00%	99.95%	99.88%	99.73%	99.62%	99.35%	99.21%	98.66%	98.15%	97.73%																																																																																																																																																												
Load balancing	100.00%	99.97%	99.93%	99.90%	99.93%	99.85%	99.89%	99.71%	99.59%	99.45%																																																																																																																																																												
In execution efficiency	100.00%	99.99%	99.95%	99.83%	99.69%	99.50%	99.31%	98.95%	98.55%	98.27%																																																																																																																																																												
Scalability for computation tasks	100.00%	99.91%	99.86%	99.80%	99.74%	99.45%	99.15%	98.71%	98.43%	97.73%																																																																																																																																																												
IPC scalability	100.00%	99.98%	100.00%	99.97%	99.96%	99.79%	99.64%	99.40%	99.24%	98.87%																																																																																																																																																												
Instruction scalability	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%																																																																																																																																																												
Frequency scalability	100.00%	99.92%	99.86%	99.83%	99.78%	99.65%	99.50%	99.30%	99.18%	98.85%																																																																																																																																																												

we get an awesome number of 0.99. The number of implicit tasks per thread is one, what we expected. Then the useful duration decreases as there are more threads, and that is good and makes sense, because each thread does less iterations. Then %Synch overhead is always 0.0 because at the time of creating the tables the histogram and the image are not generated. Then the %Sched overhead reaches a point of 4132.48, which is not terrible, because with other strategies we got worse numbers. The increase of this overhead can be due to the fact that more threads fork and maybe more probabilities exist that the threads have to wait for some other that is taking a little bit longer.

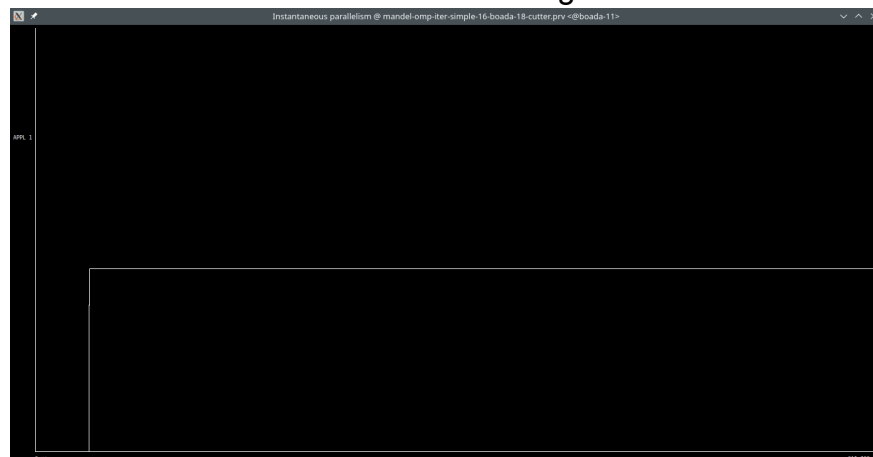
Overheads in executing implicit tasks										
Number of threads	1	2	4	8	12	16	20	24	28	32
Number of implicit tasks per thread (average us)	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0
Useful duration for implicit tasks (average us)	3208910.1	1605951.73	803331.79	401931.82	268119.27	201672.78	161827.31	135457.57	116436.17	102608.09
Load balancing for implicit tasks	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	0.99
Time in synchronization implicit tasks (average us)	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Time in fork/join implicit tasks (average us)	117.38	191.99	1023.15	1584.26	1130.8	1961.59	1608.04	3330.25	3574.28	4132.48

Table 3: Analysis done on Tue May 26 11:36:03 AM CEST 2026, par1104

Paraver analysis

· Hint Useful/Instantaneous Parallelism:

The plot is extremely useful to us, because visually we can see that all the threads are working at the same time until the end of the execution, and that is really good, as we always want the threads to work and not resting.



· Hint OpenMP/Implicit tasks in parallel constructs:

With the next plot we can see that every thread starts to work at the same time and finishes also at the same time. This means that the load balancing is extremely good and that is why we got a load balance of 99% before.



Memory analysis

THREAD 1.1.1	102,364
THREAD 1.1.2	102,152
THREAD 1.1.3	100,958
THREAD 1.1.4	101,958
THREAD 1.1.5	102,010
THREAD 1.1.6	101,510
THREAD 1.1.7	100,575
THREAD 1.1.8	102,255
THREAD 1.1.9	101,968
THREAD 1.1.10	101,156
THREAD 1.1.11	100,915
THREAD 1.1.12	102,068
THREAD 1.1.13	101,599
THREAD 1.1.14	101,256
THREAD 1.1.15	102,186
THREAD 1.1.16	102,042
Total	1,626,972
Average	101,685.75
Maximum	102,364
Minimum	100,575
StDev	538.80
Avg/Max	0.99

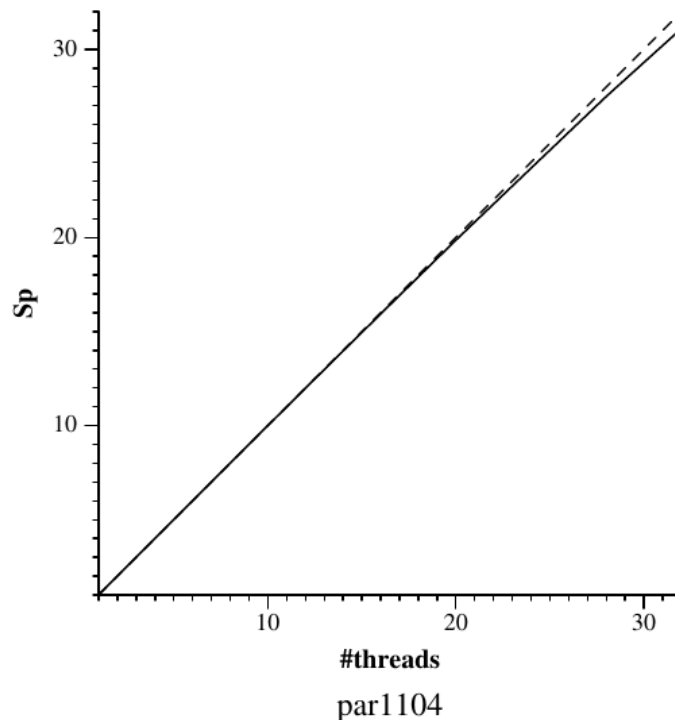
With the histogram we can see that almost every thread has 100.000 cache misses, that's extremely good because no thread is losing more time than another by making more cache misses. As always, the plot from below shows the duration of each thread, and by seeing that each thread works the same and also that everyone makes almost the same cache misses, we can see why the load balance is so good.



Then we have the l2_misses.txt file, which is:

Num threads	Fallades Cache L2	Mitjana de fallades
1	1587369	1587369
2	1591165	795582
4	1609150	402287
8	1616375	202046
12	1624117	135343
16	1631265	101954
20	1641356	82067
24	1646352	68598
28	1655110	59111
32	1651464	51608

The file tells us that even if we pass from 1 thread to 32 threads we will get almost the same cache misses. This is a really good scenary, because the threads are not adding more overhead and unnecessary time to the execution of the program.

Strong scalability

Speed-up wrt sequential time (mandel function only)

Generated by par1104 on Tue May 26 11:54:13 AM CEST 2026

With this final plot we can observe an incredible performance, we see that the real speedup is almost identical to the ideal one. This confirms the good load balance that we have been talking about before, and makes it a really good strategy to use for this Mandelbrot code.

	Number of threads					
Version	1	4	8	16	24	32
1D Block geometric data decomposition by columns	3.24	2.21	1.44	0.75	0.56	0.43
1D Block-cyclic geometric data decomposition by columns	3.25	0.83	0.43	0.23	0.16	0.14
1D Cyclic geometric data decomposition by rows	3.22	0.82	0.42	0.22	0.15	0.12

Version	Number of threads (L2 cache misses per thread)					
1D Block geometric data decomposition by columns	1613737	476589	278278	159836	121173	61903
1D Block-cyclic geometric data decomposition by columns	1612254	1152377	206053	103447	69230	51981
1D Cyclic geometric data decomposition by rows	1587369	402287	202046	101954	68598	51608
Best parallel implementation	Version		Reason why?			
	1D Cyclic geometric data decomposition by rows		Firstly we discard the first strategy, because it gives worse times and speedup. Then we have strategy 2 and 3, and even if they seem to be almost the same we choose the 3 strategy. That is due to the fact that it has a better speedup, and it would give better results if we used more threads.			